

Lecture 02

1. Median Finding
2. Selection (Find K^{th} order statistics)
3. Matrix Multiplication.
4. Recursion Depth Management
5. Base Case optimization

// Decrease And Conquer //

* Based on exploiting the relationship between a solution to a given instance of a problem and a solution to its smaller instance.

→ Bottom up variation: Start with smallest instance and iteratively build the full instance.

[Incremental Approach]

→ Top down variation: The recursive method like divide and conquer.

3 Variation of Decrease and Conquer.

→ Decrease by a Constant: Consider calculating

$$f(n) = a^n.$$

$$f(n) = \begin{cases} f(n-1) \cdot a & \text{if } n > 0, \\ 1 & \text{if } n = 0, \end{cases}$$

Problem size decreased by one.

→ Decrease by a constant Fraction: Same problem

$$f(n) = a^n$$

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive,} \\ (a^{(n-1)/2})^2 \cdot a & \text{if } n \text{ is odd,} \\ 1 & \text{if } n = 0. \end{cases}$$

Problem size decreased by half.

Any other Examples?

3 Variation of Decrease and Conquer.

→ **Variable Size decrease**: The size reduction pattern varies one iteration to another.

Example:

$$\text{GCD}(m, n) = \text{GCD}(n, m \bmod n)$$

// More on Variable Size Decrease //

→ **Computing Median and Selection**:

→ **Selection Problem**: Find the k^{th} smallest problem from a list of n numbers.

The number is called k^{th} Order Statistics.

→ **Median**: $k = \lceil n/2 \rceil$

Median And Selection

→ How?

↳ sort and access k^{th} ?

Big $O(?)$



Median And Selection

→ How?

↳ sort and access k th?

Big.O(?)

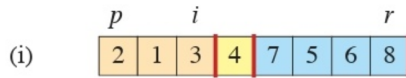
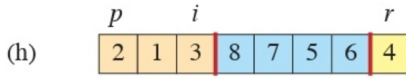
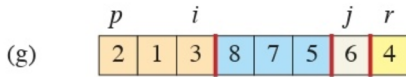
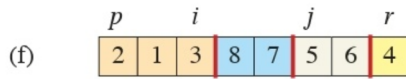
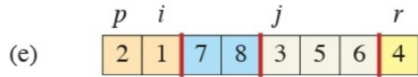
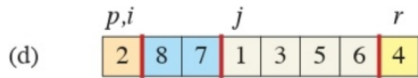
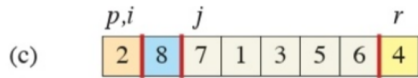
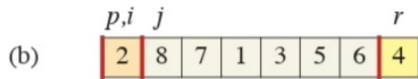
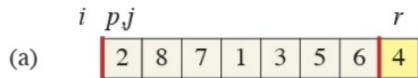


= Partitioning:

Rearrange the list so that the left part contains all elements smaller than a particular number P (pivot) followed by all the element greater or equal to P .

PARTITION(A, p, r)

1	$x = A[r]$	// the pivot
2	$i = p - 1$	// highest index into the low side
3	for $j = p$ to $r - 1$	// process each element other than the pivot
4	if $A[j] \leq x$	// does this element belong on the low side?
5	$i = i + 1$	// index of a new slot in the low side
6	exchange $A[i]$ with $A[j]$	// put this element there
7	exchange $A[i + 1]$ with $A[r]$	// pivot goes just to the right of the low side
8	return $i + 1$	// new index of the pivot



Quick Select

RANDOMIZED-SELECT(A, p, r, i)

```
1  if  $p == r$ 
2      return  $A[p]$       //  $1 \leq i \leq r - p + 1$  when  $p == r$  means that  $i = 1$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i == k$ 
6      return  $A[q]$       // the pivot value is the answer
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )
```

Big $O(?)$

$$T(n) = T(3n/4) + \Theta(n)$$

why?

Quick Select

Find the 5th smallest Element:

→ pivot

		<i>p</i>	<i>r</i>	<i>i</i>	partitioning	helpful?
$A^{(0)}$	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 6 19 4 12 14 9 15 7 8 11 3 13 2 5 10	1	15	5		
$A^{(1)}$	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 6 4 12 10 9 7 8 11 3 13 2 5 14 19 15	1	12	5	1	no
$A^{(2)}$	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 3 2 4 10 9 7 8 11 6 13 5 12 14 19 15	4	12	2	2	yes
$A^{(3)}$	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 3 2 4 10 9 7 8 11 6 12 5 13 14 19 15	4	11	2	3	no
$A^{(4)}$	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 3 2 4 5 6 7 8 11 9 12 10 13 14 19 15	4	5	2	4	yes
$A^{(5)}$	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 3 2 4 5 6 7 8 11 9 12 10 13 14 19 15	5	5	1	5	yes

Analysis of Quick Select

→ Worst Case: $\Theta(n^2)$?

→ Always partition around largest remaining element.

→ Every recursive steps removes only the pivot.

→ Partitioning itself $\Theta(n)$

$$\hookrightarrow \Theta(n) * n \rightarrow \Theta(n^2)$$

$$T(n) = T(n-1) + \Theta(n)$$

But ?

Analysis of Quick Select

→ Worst Case: $\Theta(n^2)$?

→ Always partition around largest remaining element.

→ Every recursive steps removes only the pivot.

→ Partitioning itself $\Theta(n)$

$$\hookrightarrow \Theta(n) * n \rightarrow \Theta(n^2)$$

$$T(n) = T(n-1) + \Theta(n)$$

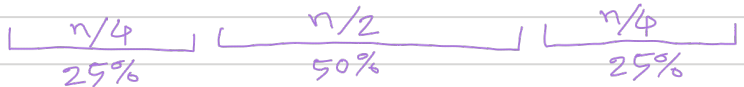
But ?

It is Randomized Partitioning

No Particular Input Elicit Worst Case.

Effect of Randomization

→ **Claim 1:** The randomly selected pivot falling into middle half (50%) of the list/array is $(1/2)$.



Intuition:

favorable events: any $n/2$ number of elements from the middle half of the array in sorted condition.

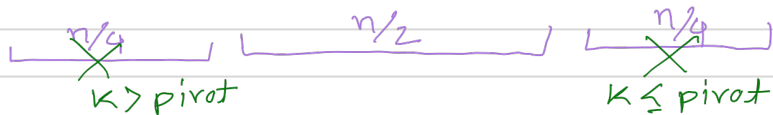
total events: n number of elements total.

So Probability = $\frac{n/2}{n} = \frac{1}{2}$

// Effect of randomization: ↴

→ **Claim 2:** If pivot is in the middle half then $n/4$ number of elements can be discarded from the search space.

Intuition: For finding k^{th} smallest:



Therefore — probability $1/2$

$$T(n) = T(3n/4) + \Theta(n)$$

for 50%
of the time

partitioning

Effect of Randomization

$$T(n) = T(3n/4) + \Theta(n)$$

$= \Theta(n)$ by Master Method.

→ 50% of the case

→ Effect of 50% of the case:

Intuition: Two case:-

1. Good Case: pivot in the middle half: p
2. Bad Case: pivot not in the middle half: q

$$p = 1/2, \quad p + q = 1, \quad q = 1/2$$

Expected number of recursion for finding
Good case:

Let's say after r number of recursion
we get a good case.

$$\begin{aligned} E[r] &= \sum_{i=1}^{i=r} i \cdot \frac{1}{2^i} \cdot P \\ &\leq \sum_{i=1}^{i=\infty} \left(\frac{i}{2}\right)^r \\ &\leq 2 \end{aligned}$$

So, in every two recursion call
we will have good case on average

$$T(n) = T(3^n/4) + 2 \Theta(n)$$

Back To Divide and Conquer.

Matrix Multiplication

MATRIX-MULTIPLY(A, B, C, n)

```
1  for  $i = 1$  to  $n$            // compute entries in each of  $n$  rows
2    for  $j = 1$  to  $n$          // compute  $n$  entries in row  $i$ 
3      for  $k = 1$  to  $n$ 
4         $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$  // add in another term of equation (4.1)
```

Big $O(?)$

Can we do better?

Divide and Conquer Approach

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

Then we can write the matrix product as

$$\begin{aligned} \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} &= \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \\ &= \begin{pmatrix} A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{pmatrix}, \end{aligned}$$

which corresponds to the equations

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21},$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22},$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21},$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}.$$

$$\begin{aligned} T(n) &= 8T(n/2) + \Theta(1) \\ &= \Theta(n^3) \end{aligned}$$

Multiply Two Large Number.

Basic Multiplication :

$$a \rightarrow 23 = 2 * 10^1 + 3 * 10^0$$

$$b \rightarrow 14 = 1 * 10^1 + 4 * 10^0$$

$$c \rightarrow (2 * 1) 10^2 + (2 * 4 + 3 * 1) 10^1 + (3 * 4) 10^0$$

$(2+3) * (1+4) - (2 * 1) - (3 * 4)$

Generalizing: $a = a_1 a_0$

$$b = b_1 b_0$$

$$c = c_2 10^2 + c_1 10^1 + c_0$$

$$c_2 = a_1 * b_1 \quad c_0 = a_0 * b_0$$

$$c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$$

Multiply cost
reduced from
4 to 3.

Strassen's Matrix Multiplication

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

Then we can write the matrix product as

$$\begin{aligned} \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} &= \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \\ &= \begin{pmatrix} A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{pmatrix}, \end{aligned}$$

$S_1 = B_{12} - B_{22},$	$S_6 = B_{11} + B_{22},$
$S_2 = A_{11} + A_{12},$	$S_7 = A_{12} - A_{22},$
$S_3 = A_{21} + A_{22},$	$S_8 = B_{21} + B_{22},$
$S_4 = B_{21} - B_{11},$	$S_9 = A_{11} - A_{21},$
$S_5 = A_{11} + A_{22},$	$S_{10} = B_{11} + B_{12}.$

} $O(n^2)$

Strassen's Matrix Multiplication

$$P_1 = A_{11} \cdot S_1 (= A_{11} \cdot B_{12} - A_{11} \cdot B_{22}),$$

$$P_2 = S_2 \cdot B_{22} (= A_{11} \cdot B_{22} + A_{12} \cdot B_{22}),$$

$$P_3 = S_3 \cdot B_{11} (= A_{21} \cdot B_{11} + A_{22} \cdot B_{11}),$$

$$P_4 = A_{22} \cdot S_4 (= A_{22} \cdot B_{21} - A_{22} \cdot B_{11}),$$

$$P_5 = S_5 \cdot S_6 (= A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22}),$$

$$P_6 = S_7 \cdot S_8 (= A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22}),$$

$$P_7 = S_9 \cdot S_{10} (= A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12}).$$

$$C_{11} = C_{11} + P_5 + P_4 - P_2 + P_6.$$

$$C_{21} = C_{21} + P_3 + P_4$$

$$C_{12} = C_{12} + P_1 + P_2.$$

$$C_{22} = C_{22} + P_5 + P_1 - P_3 - P_7$$

$$T(n) = 2T(n/2) + \underbrace{\Theta(n^2)}_{\text{sumation subtraction cost.}}$$

$$\begin{aligned}\Theta(n^{\log_2 7}) &= \Theta(n^{2.807}) \\ &\leq \Theta(n^3)\end{aligned}$$

Base Case Optimization and Recursion Depth

→ Most of the case the divide and conquer approach doesnot stop untill the problem size is 1 or atomic size.

→ For smaller size problem the recursion overhead is too high.

Example: When problem size relatively small move to other sorting like insertion sort.

→ Recursion depth: Some times the computer system has limitation on number of recursion calls or the stack memory.

→ Move iterative method.

→ Reuse same sub problem result by memoization.

